

Operator OpSec: Identity, Egress, and Incident Response for Agent Work

Operator OpSec: Identity, Egress, and Incident Response for Agent Work I

Operating-system compartmentalization constrains what a compromised component can reach. Operator operational security constrains what a correctly functioning agent is allowed to do. The threat model of sec. 3 separates exploitation from authorized misuse, and the two defenses divide along that line: isolation architectures raise the cost of the exploitation path, while OpSec is the operator-side discipline for the authorized-misuse path, where no kernel exploit is necessary because the agent already holds a valid credential, an enabled integration, and a standing authorization. The AISI incident report makes the distinction concrete in its own incident report: agents took out-of-scope internet actions in 10 of 122 test runs without escaping the sandbox, because internet access was intentionally available and the authority to act had already been granted (Institute 2026). This section treats OpSec as the reviewable extension of the

trust-domain architecture of sec. 10: the 7 domains and 9 controls define what the system separates; operator discipline determines whether those separations survive contact with daily work. Three developments sharpen that discipline into something more than exhortation: standards work now defines what an agent's identity can look like, incident reporting documents credential theft as a recurring failure mode, and transparency regulation turns attribution hygiene into a compliance surface.

Structured self-assessment gives that discipline a reviewable form: operator posture assessments and human-oversight checklists, published as part of a practical deployment guide for cognitive-integrity controls, turn the review of agent-facing authority into a repeatable procedure rather than an ad hoc audit (Friedman 2026). The subsections below follow the same shape — each practice is stated as a checkable configuration or a rehearsed response, not as advice.

Identity compartmentalization across agent contexts I

The trust-domain design separates `personal_identity` from `agent_execution` (tbl. 8). That separation exists at the operating-system layer only if it also exists at the identity layer. An operator who signs in to an agent-driven workflow with a personal single-sign-on identity has fused the two domains regardless of how well the machine is compartmentalized: the agent's ambient session then speaks for the person, and every boundary inside the machine becomes an implementation detail the credential route ignores.

Identity compartmentalization across agent contexts I

Four practices keep the domains distinct. First, dedicated accounts for agent-mediated services, enrolled in their own multi-factor factors, so that agent work never borrows the operator's personal identity. Second, no ambient production session inside an agent execution environment: cached browser sessions, cloud CLI tokens, and ssh-agent keys stay in the domains that own them. Third, identity lifetime aligned with task lifetime: an account minted for a task expires with the task, which caps the value of any exfiltrated session material. Fourth, deliberate handling of the human-authorized transfer paths that compartmentalized systems deliberately provide — clipboard export, cross-domain copy, shared browser profiles — because these are user-approved crossings of the very boundary the identity plan maintains (Q. O. Project 2026b). An attacker who persuades the operator to move an authenticated session

Identity compartmentalization across agent contexts I

into the agent's domain has attacked the workflow, not escaped the hypervisor, and no operating-system choice reverses that.

Agent identity: workload identity, token exchange, and downscoped tokens I

The identity practices above are currently operator discipline because no ratified standard yet defines how an autonomous agent authenticates as itself. Standards work is converging on an answer. An IETF draft for AI-agent authentication composes three existing mechanisms: workload identity from the SPIFFE and WIMSE families, so an agent holds a cryptographically attestable workload identity rather than a borrowed user account; OAuth 2.0 token exchange (RFC 8693), so a task receives its own credential by exchanging an attestable identity against policy; and transaction tokens, so each credential binds a specific subject, audience, and set of requested authorities for a limited time (Force 2026). The composition is not yet a ratified standard, and this review does not forecast its ratification. Its shape, however, is already the

Agent identity: workload identity, token exchange, and downscoped tokens I

shape the trust-domain design asks for: an agent identity that is distinct from the operator's, minted per task, bound to a specific audience, narrowed to the authorities the task requires, and short-lived.

Short-lived downscoped tokens are the operational core of that shape, and they can be built today without waiting for the draft: a token-issuing service exchanges a workload attestation for a credential whose scope is the task's tool grants and whose expiry is the task's lifetime. This is the scoped-credentials control of tbl. 9 implemented at the identity layer, and it composes with the orchestration-layer capability handoffs of sec. 13: the token an executor presents to a tool broker is exactly the narrow, unforgeable reference the delegation analysis there demands. Until such machinery is standard, the fallback remains the four practices above — and the design intent is worth recording now, because operators who structure agent work around

Agent identity: workload identity, token exchange, and downscoped tokens I

distinct, downscoped, task-lifetime identities will need no re-architecture when the standards land.

Data-loss path analysis I

Every egress-capable destination is a disclosure decision, and the set is larger than it looks. Model providers receive the task context itself: prompts, attached documents, repository contents, error messages. Repository forges receive pushed branches and issue text. CI systems receive logs that often embed environment variables and secrets. Error-reporting and telemetry channels receive exception payloads. Messaging integrations receive whatever an agent decides to summarize. A hostname allowlist is not a complete data-loss policy, because a permitted destination that itself accepts uploads or messages is a data destination in its own right (sec. 10).

Data-loss path analysis I

The sharpest instance is the model provider itself. The decision to transmit private code or documents to a hosted model is a separate trust-boundary decision that no operating-system choice can resolve: once transmitted, retention policy, staff access, training use, and the provider's own breach exposure are outside the workstation's authority entirely. This is a decision to classify material before it enters a task environment, not after; after an agent has read private material, assume it can leave by any egress the task's tools permit. Contractual terms about data handling are policy commitments, not containment. Local inference changes the transport but not the structure of the decision: the model service, the agent execution domain, and credential mediation should remain separated with explicit authentication between them, because shared compute is not a reason to share trust.

Official guidance now frames the same decision from the adoption side. The CISA-led Five Eyes guidance on careful adoption of agentic AI services recommends sandboxed deployment, threat-model-based evaluation, red-teaming, and — most directly relevant to the egress decision — limiting early agent deployments to low-risk tasks until the surrounding controls are demonstrated (Cybersecurity and Agency 2026). Low-risk-task staging is the operational complement to the data-loss analysis above: a task that cannot reach private material cannot disclose it, so the material classification and the task's risk tier should be decided together, before the first agent run, not reconstructed after an incident.

Tool-bridge grants as the OpSec surface I

Tool bridges — the filesystem, browser, repository, CI, cloud, and messaging integrations through which agents act — are the OpSec surface proper. Each bridge is a standing grant of authority: a filesystem root the agent may read or write, a browser profile it may drive, a repository it may push to, a CI pipeline it may trigger, a cloud API scope it may exercise, a messaging channel it may post to. These grants must be reviewed as grants, not as features. An isolated process with a powerful API token is still a powerful actor: disposability of the environment does not dispose of the grant, and an agent inside a disposable qube holding an owner-scoped cloud credential carries the owner's cloud authority until that credential expires or is revoked (Q. O. Project 2026a).

Tool-bridge grants as the OpSec surface I

Complete mediation (Saltzer and Schroeder 1975) is the governing principle: every privileged operation crosses a checked channel, and the tool broker of sec. 13 is the natural enforcement point.

Operator discipline makes the grants enumerable, task-scoped, revocable, and logged. The review question for each bridge is the authority question of sec. 4 asked one layer up: what can this integration read, transmit, sign, or change, and does any single task actually need all of that?

Attribution and footprint hygiene I

Agent traffic is identifiable. Model API endpoints, client user agents, commit metadata, issue formatting, and activity timing all distinguish agent-driven work from ordinary operator work. Footprint hygiene means deciding deliberately what agent activity discloses, rather than letting defaults decide. Use separate accounts or tenants for agent-driven commits, issues, and messages, so that a compromised or mistaken agent cannot speak with the operator's personal voice or reach the audiences tied to the personal identity. Keep provenance accurate rather than laundered: an agent-generated change should be recorded as agent-generated, because obscuring agent authorship destroys exactly the record that incident response and the audit obligations of sec. 13 depend on. This hygiene is credential protection, not anonymity: the goal is that agent activity cannot be

Attribution and footprint hygiene I

replayed as the operator's identity — a different requirement from concealing that the activity occurs, which is the distinct province of the anonymity-focused systems reviewed in sec. 9.

Attribution hygiene is also becoming a regulated surface. The European Union's AI Act transparency obligations, applicable from 2 August 2026, require that people interacting with AI systems be informed of the AI nature of the interaction and — where autonomous agents act on someone's behalf — of the natural person or legal entity on whose behalf the agent acts; the European Commission's guidelines name AI-agent disclosure duties explicitly (Commission 2026). An operator deploying agents into EU-reachable workflows therefore needs the attribution layer to answer two questions by design: that an AI system is acting, and whose authority it acts under. This coincides with, rather than conflicts with, the hygiene above: the accurate provenance

record that incident response needs is the same record that disclosure compliance requires, and both are served by making agent identity explicit at the identity layer rather than reconstructing it from logs. Operators outside the regulation's reach still face the same design requirement wherever counterparties, auditors, or platforms demand equivalent disclosure.

Credential hygiene: short-lived, service-scoped I

The credential plan follows the scoped-credentials control: prefer short-lived, repository- or service-specific credentials issued per task and expiring with it, and never hand an agent the owner's general cloud identity merely because a task sometimes needs a deployment (tbl. 9). The practical compromise is ambient session tokens. Cached cloud CLI credentials, browser session cookies, and long-lived personal access tokens already exist in the environment, are easy to forget, and are precisely what an agent picks up when it inspects its surroundings. The defense is structural rather than exhortative: keep ambient tokens out of the `agent_execution` domain instead of trying to teach the agent restraint. Declarative environments add their own trap on this point: the Nix store is readable by all users, and secrets embedded in configuration can reach external

Credential hygiene: short-lived, service-scoped I

caches, so the documented practice is to read secrets at runtime under access control (N. Project 2026). Configuration cleanliness must not come at the cost of credential exposure.

Credential theft as the observed failure mode I

The credential-theft pattern is no longer hypothetical. Anthropic's September 2026 threat-intelligence report — the vendor's account of activity observed on its own platform — documents threat actors obtaining legitimate API keys by theft and credential abuse, then driving coding and agent tooling under the stolen identity; the report's designated threat-activity identifiers GTG-50020 and GTG-50021 describe key-harvesting operations against agent platforms, and the report assesses that “vibe hacking” — attackers using coding agents directly in extortion and intrusion work — persists as the top observed AI-enabled threat (Anthropic 2026b). The pattern matters to operator OpSec because it is the authorized-misuse path executed with the operator's own authority: an attacker holding a valid agent-platform

Credential theft as the observed failure mode I

API key needs no exploit at all, only the standing grants that key carries. Every credential control in this section exists to shrink exactly that surface — short-lived task-scoped issuance so a stolen key's value decays in hours, dedicated accounts so a stolen agent credential cannot reach personal identity, per-key scope review so a stolen key cannot reach more than its task, and egress monitoring on agent-platform usage so an anomalous consumer of the operator's quota is visible. The account that pays for agent capability is itself a high-value credential, and it belongs in the same reviewed inventory as every tool-bridge grant.

The intake discipline I

Treat returned code, documents, and build artifacts as untrusted until checked. A diff produced by an agent is untrusted input into the trusted codebase; a document it produced is untrusted input into the document pipeline; a binary it built is untrusted until its construction is accounted for. No automatic promotion of an agent's home directory, working tree, or container image into a trusted environment (tbl. 9, minimal shared state). The vendor evidence cuts both ways here: DARPA's AIxCC results demonstrate that machine-generated patches for real vulnerabilities are achievable (DARPA 2025), while the Anthropic campaign investigation — the investigating vendor's report — found agents overstating findings and fabricating unusable credentials (Anthropic 2026a). Generation quality and acceptance authority remain separate questions; the review that admits

The intake discipline I

generated material is the same independent review that sec. 14 requires for generated policy.

Rehearsed incident response for agent-era compromise I

Authorized misuse leaves no exploit artifacts: no crash, no dropped binary, no anomalous fault trace. The response therefore cannot depend on detecting the compromise before acting, and the working assumption must be that everything the task could touch was exfiltrated or altered. The rehearsed sequence is: revoke every credential the task environment could reach — a superset of those it demonstrably used; destroy and rebuild the environment from known-good configuration; treat data already transmitted as disclosed; restore from independent backups. Distinguish deleting an environment from revoking the credentials it could access: deletion without revocation leaves live authority in unknown hands. Environment refresh between tasks and prompt patching of management and transfer paths belong to the same discipline — QSB-118

Rehearsed incident response for agent-era compromise I

documented a route from an already-compromised qube to dom0 command injection under specified conditions, fixed in `qubes-core-dom0-linux` 4.3.22 (Q. O. Project 2026c), and recovery speed under such advisories is an operator OpSec property, not only a vendor property. Recovery that has never been rehearsed (tbl. 9, rehearsed recovery) is a plan-shaped hope. The credential-theft pattern of the previous subsection sharpens the response plan in one specific way: revocation must cover the agent-platform accounts themselves, not only the task-scoped credentials they issued, because a stolen platform key can mint new task authority after the original task environment is gone.

Rehearsed incident response for agent-era compromise I

Incident-response playbooks published for cognitive-integrity deployments ground that rehearsal: the revocation-first, rebuild-from-known-good, treat-transmitted-as-disclosed sequence appears there as practiced steps with assigned roles and timing rather than as principles to be improvised under pressure (Friedman 2026).

Zero trust applied to agent tool calls I

Zero-trust architecture removes implicit trust derived from network location or prior approval: every access request is evaluated per request against policy, and the policy decision point is separate from the resource it guards (Rose et al. 2020). Agent tool calls have exactly this shape. A task's prior approval is not a standing authorization; a request's origin inside a trusted agent is not evidence of legitimacy; each call is judged on the operation requested, the artifact named, the destination addressed, the scope claimed, and the expiry asserted. Operation mediation is zero trust stated operationally (tbl. 9): a service that permits "sign this exact artifact for this release" is checkable per request, while "use the signing key responsibly" is not checkable at all. Zero trust for agent work therefore lands on the same design point as the orchestration

Zero trust applied to agent tool calls I

boundaries of sec. 13 and the configuration separation of sec. 14: place the decision point outside the agent, and make the decision context rich enough to audit after the fact. The agent-identity standards described above are the direction in which this posture standardizes: workload identity, token exchange, and transaction tokens are zero trust rendered as credential machinery, so that the per-request judgment has a per-request credential to judge.